HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA VIỄN THÔNG 1

BÁO CÁO BÀI TẬP LỚN
KỸ THUẬT LẬP TRÌNH

Chủ đề: Quản lý dịch vụ truyền hình IPTV

GV hướng dẫn: Nguyễn Hữu Cầm

Các thành viên:

1,	Nguyễn Quốc Việt (nhóm trưởng)	B24DCVT395
2,	Dương Đức Mạnh	B24DCVT240
3,	Đỗ Gia Huy	B24DCVT172
4,	Trần Thanh Tùng	B24DCVT383
5,	Nguyễn Bá Thành Văn	B24DCVT389

HÀ NỘI - 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU 2
1.1, Dịch vụ truyền hình IPTV 2
1.2, Các đối tượng cần quản lý 2
1.3, Các chức năng 2
CHƯƠNG 2: MÔI TRƯỜNG & CÔNG CỤ LẬP TRÌNH 3
2.1. Môi trường phát triển và trình biên dịch 3
2.2. Quy trình làm việc nhóm 3
2.3. Danh mục thư viện chuẩn (STL) và mục tiêu sử dụng 3
CHƯƠNG 3: THIẾT KẾ GIẢI PHÁP HƯỚNG ĐỐI TƯỢNG 4
3.1. Kiến trúc phân tầng và giải pháp OOP 4
3.2. Vận dụng 4 đặc trưng cốt lõi của OOP 5
3.3. Thiết kế cơ chế lưu trữ 6
3.5. Quản lý rủi ro, bẫy lỗi và đảm bảo tính toàn vẹn dữ liệu 7
CHƯƠNG 4: NHỮNG KIẾN THỨC TỰ TÌM HIỂU 8
4.1. Lập trình tổng quát với Class Template Repository<T> 8
4.2. Cơ chế kiểm soát và xử lý ngoại lệ (Exception Handling) 9
CHƯƠNG 5: TỔNG KẾT & HƯỚNG PHÁT TRIỂN 10
5.1. Kết quả đạt được 10
5.2. Hạn chế còn tồn tại 10
5.3. Hướng phát triển trong tương lai 10
BẢNG PHÂN CÔNG 11
ĐẶC TẢ USE-CASE 12

CHƯƠNG 1: GIỚI THIỆU

1.1, Dịch vụ truyền hình IPTV

IPTV (Internet Protocol Television - Truyền hình giao thức Internet) là dịch vụ phát sóng truyền hình sử dụng giao thức Internet qua mạng băng thông rộng (thay vì dùng cáp đồng trục hay ăng-ten truyền thống). IPTV vượt trội hơn các phương thức phát sóng truyền thống ở chất lượng truyền dẫn, độ ổn định và khả năng kết nối hai chiều, cho phép thuê bao truy cập vào các nội dung theo yêu cầu như Xem lại hay xem video theo yêu cầu (VOD/Video on demand).

Mục tiêu của nhóm 10 trong việc xây dựng phần mềm này là số hoá toàn bộ quy trình quản lý hệ thống truyền hình nhằm đảm bảo tính toàn vẹn của dữ liệu và hỗ trợ thao tác nghiệp vụ nhanh chóng.

1.2, Các đối tượng cần quản lý

+ Thể loại kênh & Nội dung VOD

+ Kênh truyền hình & Lịch phát sóng

+ Hoá đơn & Hợp đồng

+ Gói kênh & Khách hàng

+ Set-top-box & Phiếu hỗ trợ khách hàng

1.3, Các chức năng

C (Create - Thêm mới): Nhập dữ liệu từ bàn phím, kiểm tra tính hợp lệ (mã không trùng, chuỗi không rỗng, số không âm, khóa ngoại phải tồn tại) và tự động lưu vào file .txt.

R (Read - Xem & Tìm kiếm):

Hiển thị: Xuất danh sách đối tượng dưới dạng bảng console có tiêu đề cột và căn lề thẳng hàng (setw).

Tìm kiếm: Tra cứu nhanh theo mã định danh hoặc tên/tiêu đề (không phân biệt chữ hoa, chữ thường).

U (Update - Cập nhật): Tìm đối tượng theo mã định danh duy nhất (khóa chính được cố định, không cho phép sửa), cho phép chỉnh sửa từng trường thông tin (bấm Enter để giữ nguyên giá trị cũ) và cập nhật lại file.

D (Delete - Xóa): Tìm theo mã, hiển thị hộp thoại xác nhận (Y/N) trước khi xóa và kiểm tra ràng buộc toàn vẹn dữ liệu (chặn xóa nếu bản ghi đang được thực thể khác tham chiếu).

CHƯƠNG 2: MÔI TRƯỜNG & CÔNG CỤ LẬP TRÌNH

2.1. Môi trường phát triển và trình biên dịch

- **Hệ điều hành thử nghiệm:** Microsoft Windows 11 (64-bit).
- **Trình biên dịch:** GNU Compiler Collection (g++) v6.3.0
- **IDE:** DevC++ v6.3, Visual Studio Code (C/C++ Extension Pack) v1.137.0

2.2. Quy trình làm việc nhóm

Để hoàn thành dự án gồm 5 thành viên với nhiều đầu việc song song, nhóm đã áp dụng quy trình kiểm soát phiên bản hiện đại:

1. **Khởi tạo khung:** Trưởng nhóm tạo repository trên GitHub, định nghĩa sẵn lớp cơ sở Entity.h, lớp khuôn mẫu Repository.h và cấu trúc thư mục dữ liệu data/.
2. **Đồng bộ hóa & Tích hợp:** Sau khi hoàn thành và tự kiểm thử cục bộ, thành viên trực tiếp commit về nhánh main. Trưởng nhóm thực hiện rà soát mã, sửa lỗi và hợp nhất vào chương trình chung.

2.3. Danh mục thư viện chuẩn (STL) và mục tiêu sử dụng

Dự án sử dụng hoàn toàn thư viện chuẩn của C++ để đảm bảo tính độc lập, không phụ thuộc thư viện bên ngoài:

STT	Thư viện	Mục tiêu kỹ thuật cụ thể
1	`<iostream>`	Cung cấp đối tượng luồng vào/ra (cin, cout, cerr). Dùng để xây dựng giao diện Console, in thông báo hướng dẫn và tiếp nhận lựa chọn menu.
2	`<string>`	Quản lý chuỗi ký tự động (std::string). Hỗ trợ thao tác chuỗi linh hoạt, tự quản lý bộ nhớ đệm, dùng cho toàn bộ thuộc tính văn bản.

STT	Thư viện	Mục tiêu kỹ thuật cụ thể
3	`<vector>`	Cấu trúc dữ liệu mảng động (std::vector<T>) trong thư viện mẫu STL. Dùng làm cấu trúc dữ liệu chính trong Repository<T> để nạp danh sách thực thể vào RAM.
4	`<fstream>`	Quản lý luồng tệp tin (ifstream, ofstream). Phục vụ việc đọc/ghi dữ liệu bền vững ra các tệp tin .txt theo cơ chế tự động mở và đóng an toàn.
5	`<sstream>`	Cung cấp stringstream và ostringstream. Đóng vai trò bộ phân tích cú pháp (Parser) để chuyển đổi giữa đối tượng C++ và dòng văn bản phân cách bởi ký tự |.
6	`<iomanip>`	Cung cấp các thao tác định dạng luồng (setw, left, right, setprecision, fixed). Dùng để in bảng dữ liệu căn hàng thẳng cột trên console.
7	`<algorithm>`	Cung cấp thuật toán tối ưu sẵn có: std::find_if (tìm kiếm phần tử theo vị trí), std::remove_if (xóa phần tử), std::transform (chuyển đổi chuỗi thường/hoa).
8	`<stdexcept>`	Cung cấp cây phân cấp ngoại lệ chuẩn (invalid_argument, runtime_error, out_of_range). Dùng để bẫy lỗi logic nghiệp vụ và dữ liệu sai quy cách.
9	`<limits>`	Cung cấp std::numeric_limits<streamsize>::max(). Dùng để làm sạch bộ đệm bàn phím sau khi dùng toán tử cin >>, ngăn chặn hiện tượng trôi lệnh khi dùng getline.
10	`<cstdlib>`	Cung cấp hàm system("cls") hoặc system("clear") để làm mới màn hình terminal khi chuyển đổi giữa các menu con.

CHƯƠNG 3: THIẾT KẾ GIẢI PHÁP HƯỚNG ĐỐI TƯỢNG

3.1. Kiến trúc phân tầng và giải pháp OOP

Hệ thống được thiết kế theo mô hình 3 tầng (3-Layer Architecture) thu nhỏ, đảm bảo nguyên tắc phân tách trách nhiệm (Separation of Concerns):

1. **Tầng Thực thể (Entity Layer):** Lớp trừu tượng Entity định nghĩa giao diện chung cho tất cả dữ liệu. 10 lớp cụ thể kế thừa và hiện thực hóa các phương thức này.
2. **Tầng Truy xuất & Lưu trữ (Data Access Layer - Repository Pattern):** Lớp khuôn mẫu Repository<T> chịu trách nhiệm quản lý bộ nhớ đệm và giao tiếp với hệ thống tệp tin. Các lớp thực thể không cần biết dữ liệu được lưu vào file như thế nào.
3. **Tầng Giao diện & Điều khiển (Presentation & Control Layer):** File main.cpp và các hàm menu đóng vai trò điều hướng, tiếp nhận thao tác người dùng, gọi tầng Repository xử lý và hiển thị thông báo.

3.2. Vận dụng 4 đặc trưng cốt lõi của OOP

1. Tính Đóng gói (Encapsulation)

Toàn bộ các thuộc tính trong 10 lớp thực thể đều được đặt ở mức truy cập private (hoặc protected). Bên ngoài không thể can thiệp hay sửa đổi trực tiếp dữ liệu mà bắt buộc phải thông qua các phương thức Getter và Setter có kiểm tra tính hợp lệ:

```
cpp
void NoiDungVOD::setThoiLuong(int tl) {
    if (tl <= 0) {
        throw invalid_argument("Thoi luong phai la so nguyen duong (phut)!");
    }
    this->thoiLuong = tl;
}
```

2. Tính Trừu tượng (Abstraction)

Lớp Entity đóng vai trò là một lớp trừu tượng (Abstract Class) chứa các hàm thuần ảo (pure virtual functions gán = 0). Lớp này che giấu toàn bộ chi tiết xử lý bên dưới, chỉ quy định bộ khung hợp đồng mà mọi đối tượng trong hệ thống bắt buộc phải có.

3. Tính Kế thừa (Inheritance)

Tất cả 10 đối tượng nghiệp vụ đều kế thừa công khai (public Entity) từ lớp cơ sở Entity. Điều này giúp tái sử dụng cấu trúc giao tiếp chung, đồng thời mở đường cho việc áp dụng tính đa hình trong toàn bộ ứng dụng.

4. Tính Đa hình (Polymorphism)

Tính đa hình được thể hiện rõ nét qua việc gọi các hàm ảo thông qua con trỏ hoặc tham chiếu Entity. Điển hình là hàm hiển thị bảng dùng chung:

```
cpp
template <typename T>
void displayTable(const vector<T>& items) {
    if (items.empty()) {
        cout << "\n  [Danh sach trong!]\n";
        return;
    }
    items[0].displayTableHeader(); // Goi da hinh toi tieu de
bang cua lop con
    for (size_t i = 0; i < items.size(); ++i) {
        items[i].displayTableRow((int)i + 1);
    }
}
```

3.3. Thiết kế cơ chế lưu trữ

Dữ liệu của hệ thống được lưu trữ trong thư mục data/ với mỗi đối tượng tương ứng với một tệp .txt riêng biệt (ví dụ: data/theloaikenh.txt, data/noidungvod.txt).

- **Quy ước định dạng:** Các trường dữ liệu trong một bản ghi được phân tách bằng ký tự gạch đứng (|). Ký tự này không xuất hiện trong nội dung văn bản thông thường, giúp quá trình phân tích (parse) không bị nhầm lẫn.
- **Cơ chế nạp tự động (Load on Startup):** Khi khởi tạo đối tượng Repository<T>, hàm khởi tạo (Constructor) sẽ kiểm tra sự tồn tại của tệp. Nếu tệp chưa tồn tại, hệ thống tự động tạo tệp rỗng mới. Nếu tệp đã có dữ liệu, từng dòng được đọc qua getline, gọi hàm fromFileString để tái tạo đối tượng đưa vào std::vector<T>.
- **Cơ chế ghi tự động (Save on Change):** Mỗi khi có thao tác làm thay đổi trạng thái (Thêm bản ghi mới, Sửa bản ghi, Xóa bản ghi), phương thức saveToFile() được gọi tức thì để đồng bộ hóa toàn bộ danh sách trong RAM xuống ổ cứng.

3.5. Quản lý rủi ro, bẫy lỗi và đảm bảo tính toàn vẹn dữ liệu

Trong quá trình xây dựng hệ thống, nhóm đã xác định và giải quyết triệt để các rủi ro hệ thống:

1. **Rủi ro trôi lệnh khi nhập liệu:** Khi chuyển từ nhập số (cin >> x) sang nhập chuỗi (getline), ký tự \n còn sót lại trong bộ đệm sẽ khiến lệnh đọc chuỗi bị bỏ qua. Nhóm đã xử lý triệt để bằng cin.ignore(numeric_limits<streamsize>::max(), '\n').
2. **Rủi ro vi phạm toàn vẹn khóa ngoại (Foreign Key Integrity):**
    - *Khi thêm mới:* Trước khi thêm một NoiDungVOD, hệ thống quét repoTheLoai xem maTheLoai nhập vào có tồn tại hay không. Nếu không, lập tức từ chối thao tác.
    - *Khi xóa:* Trước khi xóa một TheLoaiKenh, hệ thống kiểm tra toàn bộ danh sách NoiDungVOD. Nếu phát hiện có nội dung đang thuộc thể loại này, thao tác xóa bị chặn và hiển thị thông báo yêu cầu chuyển thể loại của các video liên quan trước.
3. **Rủi ro trùng lặp khóa chính:** Trong phương thức Repository<T>::add, một vòng lặp quét kiểm tra trùng getId(). Nếu mã đã tồn tại, hàm trả về false và từ chối ghi đè.

3.6. Tiềm năng mở rộng và nâng cấp hệ thống

Kiến trúc mã nguồn mở đường cho các bước phát triển kế tiếp:

- **Chuyển đổi nguồn dữ liệu:** Chỉ cần viết lại phần thân hàm loadFromFile() và saveToFile() trong Repository<T> sang kết nối cơ sở dữ liệu quan hệ (SQLite, MySQL) mà không cần thay đổi dù chỉ một dòng code nghiệp vụ trong các lớp thực thể.
- **Phát triển giao diện đồ họa (GUI):** Tầng điều khiển có thể dễ dàng chuyển đổi từ console sang thư viện đồ họa như Qt hoặc wxWidgets.

CHƯƠNG 4: NHỮNG KIẾN THỨC TỰ TÌM HIỂU

4.1. Lập trình tổng quát với Class Template Repository<T>

- **Vấn đề đặt ra:** Hệ thống có 10 thực thể. Nếu mỗi thực thể đều phải viết một lớp riêng để đọc file, ghi file, tìm kiếm và xóa phần tử, chương trình sẽ có hàng nghìn dòng code trùng lặp (Boilerplate code), cực kỳ khó bảo trì và dễ sinh lỗi.
- **Kiến thức tự học:** Lập trình mẫu (Template Metaprogramming) trong C++. Cách định nghĩa một lớp tổng quát nhận tham số kiểu T, trong đó T đại diện cho bất kỳ lớp nào hiện thực hóa giao diện Entity.
- **Hiện thực hóa trong dự án:**

```
cpp
template <typename T>
class Repository {
private:
    vector<T> items;
    string fileName;
public:
    Repository(const   string&   file)   :   fileName(file)   {
loadFromFile(); }

    bool add(const T& item) {
        for (const auto& existing : items) {
            if (existing.getId() == item.getId()) return false;
        }
        items.push_back(item);
        saveToFile();
        return true;
    }
    // Các thao tác findById, remove, search, getAll...
};
```

- **Lợi ích mang lại:** Tái sử dụng 100% logic quản lý dữ liệu cho cả 10 use-case. Giảm hơn 70% lượng mã nguồn dư thừa, tập trung kiểm soát lỗi tệp tin tại một nơi duy nhất.

4.2. Cơ chế kiểm soát và xử lý ngoại lệ (Exception Handling)

- **Vấn đề đặt ra:** Khi người dùng nhập dữ liệu sai (chuỗi rỗng, giá tiền âm, định dạng ngày sai lệch), các chương trình C++ căn bản thường sử dụng giá trị trả về (return -1, return false) hoặc thoát đột ngột (exit(1)). Cách làm này khiến code bị phân mảnh bởi hàng loạt khối lệnh if/else lồng nhau.
- **Kiến thức tự học:** Cơ chế ném và bắt ngoại lệ chuẩn trong C++ (try, catch, throw), phân cấp thư viện <stdexcept> và nguyên lý an toàn biệt lệ (Exception Safety).
- **Hiện thực hóa trong dự án:**
    - Tại các hàm setter của từng lớp con: Nếu dữ liệu không thỏa mãn, chủ động throw invalid_argument("Mô tả lỗi cụ thể").
    - Tại giao diện điều khiển main.cpp: Toàn bộ các bước nhập liệu được bao bọc trong khối try/catch. Khi có ngoại lệ xảy ra, thông báo lỗi được in ra màu đỏ/cảnh báo và hệ thống tiếp tục hoạt động mà không bị crash:

```
cpp
try {
    TheLoaiKenh tl;
    tl.inputData();
    if (repoTheLoai.add(tl)) {
        cout << "\n  >> Them the loai kenh thanh cong!\n";
    }
} catch (const invalid_argument& e) {
    cout << "\n  [LOI DU LIEU] " << e.what() << "\n";
} catch (const exception& e) {
    cout << "\n  [LOI HE THONG] " << e.what() << "\n";
}
```

CHƯƠNG 5: TỔNG KẾT & HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

1. **Hoàn thành 100% mục tiêu đề bài:** Triển khai đầy đủ 10/10 use-case phân công cho 5 thành viên, mỗi use-case đều thực hiện trọn vẹn 5 chức năng CRUD.
2. **Tuân thủ xuất sắc các nguyên lý OOP:** Thiết kế hướng đối tượng mạch lạc với lớp cha Entity, tận dụng triệt để tính đa hình và đóng gói an toàn.
3. **Chương trình hoạt động mượt mà:** Khả năng xử lý nhập liệu an toàn, bẫy lỗi toàn diện, lưu trữ dữ liệu vĩnh viễn vào các file văn bản độc lập.
4. **Phối hợp nhóm hiệu quả:** Thể hiện rõ tinh thần làm việc nhóm qua việc phân chia trách nhiệm, chia sẻ thư viện dùng chung và hợp nhất mã nguồn qua Git.

5.2. Hạn chế còn tồn tại

- Cơ chế tìm kiếm hiện tại dựa trên so khớp tuyến tính (Linear Search O(n)), phù hợp với bộ dữ liệu vừa và nhỏ dưới 10.000 bản ghi, nhưng cần cấu trúc chỉ mục (Indexing) nếu dữ liệu lên đến hàng triệu bản ghi.

5.3. Hướng phát triển trong tương lai

- Tích hợp cơ sở dữ liệu SQLite nhúng trực tiếp vào ứng dụng để tăng tốc độ truy vấn và hỗ trợ giao dịch (Transaction).
- Xây dựng giao diện ứng dụng để bàn (Desktop GUI) sử dụng thư viện Qt hoặc phát triển API Backend để kết nối với Web/Mobile App.
- Bổ sung cơ chế phân quyền người dùng: Tài khoản Quản trị viên (Admin), nhân viên kỹ thuật, khách hàng (client).

BẢNG PHÂN CÔNG

Tên nhóm: Nhóm 10

Đề tài: Quản lý dịch vụ truyền hình IPTV

Lớp: TEL1340 Giảng viên hướng dẫn: Nguyễn Hữu Cầm

STT	Họ và tên	MSSV	Vai trò	Use case 1	Use case 2	Đóng góp (%)
1	Nguyễn Quốc Việt	B24DCVT395	Trưởng nhóm	Thể loại kênh	Nội dung VOD	25
2	Dương Đức Mạnh		Thành viên	Kênh truyền hình	Lịch phát sóng	18.75
3	Nguyễn Bá Thành Văn		Thành viên	Set-top-box	Phiếu hỗ trợ khách hàng	18.75
4	Đỗ Gia Huy		Thành viên	Hoá đơn	Hợp đồng	18.75
5	Trần Thanh Tùng		Thành viên	Gói kênh	Khách hàng	18.75

Ghi chú: cột Đóng góp do cả nhóm thống nhất, tổng bằng 100%. Nhóm 4 người để trống dòng 5.

ĐẶC TẢ USE-CASE

Mục	Nội dung
Mã use case	UC01
Tên use case	Quản lý thể loại kênh
Sinh viên thực hiện	Họ tên: Nguyễn Quốc Việt - MSSV: B24DCVT395
Lớp đối tượng (C++)	TheLoaiKenh (kèm lớp menu TheLoaiKenhMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maTheLoai, tenTheLoai, moTa
Tên file dữ liệu	data/theloaikenh.txt
Định dạng một bản ghi	maTheLoai|tenTheLoai|moTa
Tham chiếu tới (khóa ngoại)	Không có
Được tham chiếu bởi	NoiDungVOD (maTheLoai), KenhTruyenHinh (maTheLoai)
Chức năng Thêm (C)	Nhập mã, tên và mô tả thể loại. Kiểm tra trùng mã khóa chính qua Repository::add.
Chức năng Xem, tìm kiếm, sắp xếp (R)	+ Xem toàn bộ danh sách định dạng bảng qua displayTable(). + Tìm kiếm không phân biệt hoa thường theo mã thể loại hoặc tên thể loại.
Chức năng Cập nhật (U)	Tìm kiếm theo mã thể loại; hiển thị dữ liệu hiện tại; cho phép nhấn Enter để giữ nguyên giá trị cũ; gọi repo.saveAfterUpdate().
Chức năng Xóa (D)	Tìm theo mã; kiểm tra toàn vẹn dữ liệu: không cho xóa nếu đang có nội dung VOD hoặc Kênh truyền hình tham chiếu đến mã thể loại này; xác nhận y/n trước khi xóa.
Kỹ thuật OOP áp dụng	Trừu tượng hóa (Interface Entity), Kế thừa, Đóng gói (private data, setter validate), Đa hình (ghi đè 9 hàm ảo), Generic Programming (Repository<TheLoaiKenh>).

Mục	Nội dung
Mã use case	UC02
Tên use case	Quản lý ……
Sinh viên thực hiện	Họ tên: …… MSSV: ……
Lớp đối tượng (C++)	NoiDungVOD (kèm lớp menu NoiDungVODMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maVOD (string), tieuDe (string), maTheLoai (string), thoiLuong (int), namSanXuat(int), daoDien (string), moTa (string)
Tên file dữ liệu	data/noidungvod.txt
Định dạng một bản ghi	maVOD|tieuDe|maTheLoai|thoiLuong|namSanXuat|daoDien|moTa
Tham chiếu tới (khóa ngoại)	TheLoaiKenh (maTheLoai)
Được tham chiếu bởi	Không có
Chức năng Thêm (C)	Hiển thị danh mục thể loại; kiểm tra mã thể loại có tồn tại không; kiểm tra ràng buộc số nguyên và năm sản xuất; thêm vào repository.
Chức năng Xem, tìm kiếm, sắp xếp (R)	+ Xem bảng danh sách VOD chuẩn căn lề setw. + Tìm kiếm gần đúng theo mã VOD hoặc tiêu đề phim.
Chức năng Cập nhật (U)	Tìm theo mã; hiển thị thông tin cũ; cho phép giữ nguyên bằng Enter; kiểm tra mã thể loại mới nhập vào; có cơ chế sao lưu phục hồi (rollback) nếu lỗi.
Chức năng Xóa (D)	Tìm theo mã VOD; hiển thị thông tin chi tiết; xác nhận người dùng (Y/N); xóa khỏi danh sách và lưu lại file.
Kỹ thuật OOP áp dụng	Trừu tượng hóa, Kế thừa, Đóng gói, Đa hình runtime, Kiểm soát toàn vẹn khóa ngoại (Referential Integrity), Template Repository.

Mục	Nội dung
Mã use case	UC03
Tên use case	Quản lý kênh truyền hình
Sinh viên thực hiện	Họ tên: Dương Đức Mạnh MSSV: B24DCVT240
Lớp đối tượng (C++)	KenhTruyenHinh (kèm lớp menu KenhTruyenHinhMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maKenh(string), tenKenh(string), maTheLoai(string), chatLuong (string), ngonNgu(string), moTa (string)
Tên file dữ liệu	data/kenhtruyenhinh.txt
Định dạng một bản ghi	maKenh|tenKenh|maTheLoai|chatLuong|ngonNgu|moTa
Tham chiếu tới (khóa ngoại)	TheLoaiKenh (maTheLoai)
Được tham chiếu bởi	LichPhatSong (maKenh)
Chức năng Thêm (C)	Kiểm tra mã thể loại tồn tại trong repoTheLoai; kiểm tra trùng mã kênh; thêm mới vào kho lưu trữ.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Xem toàn bộ danh sách kênh dạng bảng, tìm kiếm theo mã kênh hoặc tên kênh.
Chức năng Cập nhật (U)	Nhập mã kênh; cập nhật tên, thể loại, chất lượng, ngôn ngữ; kiểm tra hợp lệ của mã thể loại mới.
Chức năng Xóa (D)	Chặn xóa nếu kênh đang có chương trình trong Lịch phát sóng (repoLichPS).
Kỹ thuật OOP áp dụng	Đóng gói, kế thừa, đa hình qua phương thức ảo displayTableHeader/displayTableRow, kiểm tra toàn vẹn liên kết thực thể.

Mục	Nội dung
Mã use case	UC04
Tên use case	Quản lý lịch phát sóng
Sinh viên thực hiện	Họ tên:Dương Đức Mạnh MSSV: B24DCVT240
Lớp đối tượng (C++)	LichPhatSong (kèm lớp menu LichPhatSongMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maLich(string), maKenh(string), tenChuongTrinh(string), ngayPhatSong(string), gioBatDau(string), gioKetThuc(string) (định dạng HH:MM), moTa(string)
Tên file dữ liệu	data/lichphatsong.txt
Định dạng một bản ghi	maLich|maKenh|tenChuongTrinh|ngayPhatSong|gioBatDau|gioKetThuc|moTa
Tham chiếu tới (khóa ngoại)	kenhTruyenHinh (maKenh)
Được tham chiếu bởi	Không có
Chức năng Thêm (C)	Kiểm tra kênh truyền hình tồn tại; kiểm tra định dạng ngày giờ và thứ tự logic khung giờ.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Xem bảng lịch phát sóng chi tiết, tìm kiếm theo mã lịch hoặc tên chương trình.
Chức năng Cập nhật (U)	Sửa thông tin phát sóng, khung giờ phát (Enter để giữ nguyên).
Chức năng Xóa (D)	Xóa lịch phát sóng theo mã xác định với xác nhận Y/N.
Kỹ thuật OOP áp dụng	Trừu tượng hóa, Đa hình, Đóng gói thuật toán kiểm tra ngày giờ nội tại trong class.

Mục	Nội dung
Mã use case	UC05
Tên use case	Quản lý Gói kênh truyền hình
Sinh viên thực hiện	Họ tên: Trần Thanh Tùng MSSV: ……
Lớp đối tượng (C++)	GoiKenh (kèm lớp menu GoiKenhMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maGoi (string), tenGoi (string), moTa (string), giaThue (double), soKenh (int), loaiGoi (string), trangThai (string)
Tên file dữ liệu	data/goi_kenh.txt
Định dạng một bản ghi	maGoi|tenGoi|moTa|giaThue|soKenh|loaiGoi|trangThai
Tham chiếu tới (khóa ngoại)	Không có
Được tham chiếu bởi	KhachHang (maGoiKenh), HopDong (goiDichVu)
Chức năng Thêm (C)	Nhập liệu an toàn qua InputHelper; kiểm tra giá tiền và loại gói; chống trùng lặp mã gói.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Xem danh sách dạng bảng, Tìm kiếm theo mã, tên, loại gói hoặc khoảng giá thuê, Sắp xếp danh sách theo tên A-Z, giá tăng/giảm dần, số kênh.
Chức năng Cập nhật (U)	Cập nhật thông tin chi tiết gói kênh; ghi đè an toàn xuống file.
Chức năng Xóa (D)	Ràng buộc nghiệp vụ: Kiểm tra qua callback Lambda ktraGoi, chặn xóa nếu đang có Khách hàng sử dụng gói cước này.
Kỹ thuật OOP áp dụng	Kế thừa Entity, Đóng gói, Đa hình, Lập trình hàm (Lambda Callback / Dependency Injection).

Mục	Nội dung
Mã use case	UC06
Tên use case	Quản lý khách hàng
Sinh viên thực hiện	Họ tên: Trần Thanh Tùng MSSV: ……
Lớp đối tượng (C++)	KhachHang (kèm lớp menu KhachHangMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	• maKH (string), hoTen (string), soCCCD (string), soDienThoai (string), email (string), diaChi (string), ngayDangKy (string), maGoiKenh (string), trangThai (string)
Tên file dữ liệu	data/khach_hang.txt
Định dạng một bản ghi	maKH|hoTen|soCCCD|soDienThoai|email|diaChi|ngayDangKy|maGoiKenh|trangThai
Tham chiếu tới (khóa ngoại)	GoiKenh (maGoiKenh)
Được tham chiếu bởi	SetTopBox (maKH), HopDong (maKhachHang)
Chức năng Thêm (C)	Kiểm tra tính hợp lệ của CCCD, SĐT, Email; cảnh báo nếu mã gói kênh không có trong hệ thống; thêm mới.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Xem danh sách khách hàng, Tìm kiếm đa năng: theo mã, họ tên, số điện thoại, gói kênh, trạng thái, Sắp xếp theo họ tên A-Z, mã KH, ngày đăng ký, Thống kê số lượng khách hàng theo trạng thái và theo từng gói kênh (dùng std::map).
Chức năng Cập nhật (U)	Tìm khách hàng theo mã; cập nhật thông tin cá nhân và gói cước.
Chức năng Xóa (D)	Tìm theo mã; cảnh báo nếu khách hàng đang ở trạng thái "Hoat dong"; xác nhận Y/N.
Kỹ thuật OOP áp dụng	Trừu tượng hóa, Đóng gói, Đa hình, Sử dụng cấu trúc dữ liệu ánh xạ std::map kết hợp OOP.

Mục	Nội dung
Mã use case	UC07
Tên use case	Quản lý set-top-box
Sinh viên thực hiện	Họ tên: Nguyễn Bá Thành Văn MSSV: ……
Lớp đối tượng (C++)	SetTopBox (kèm lớp menu SetTopBoxMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maSTB (string), serial (string), model (string), maKH (string), trangThai (int) (1: Ton kho, 2: Dang su dung, 3: Hong)
Tên file dữ liệu	data/settopbox.txt
Định dạng một bản ghi	maSTB|serial|model|maKH|trangThai
Tham chiếu tới (khóa ngoại)	KhachHang (maKH)
Được tham chiếu bởi	PhieuHoTro (maSTB)
Chức năng Thêm (C)	Nhập thông tin thiết bị; kiểm tra mã khách hàng hợp lệ nếu được gán ngay; kiểm tra trùng mã STB.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Hiển thị bảng danh sách kho thiết bị kèm trạng thái dạng chữ trực quan, Tìm kiếm theo mã STB, số serial hoặc model máy.
Chức năng Cập nhật (U)	Cập nhật serial, model, gán khách hàng mới hoặc hủy gán (nhập '-'), đổi trạng thái thiết bị.
Chức năng Xóa (D)	**Ràng buộc an toàn:** Chặn xóa nếu thiết bị đang được sử dụng (trangThai == 2) hoặc còn phiếu hỗ trợ kỹ thuật liên quan (coPhieu).
Kỹ thuật OOP áp dụng	Đóng gói, Kế thừa, Đa hình, Dependency Injection (truyền kiểm tra logic khTonTai, coPhieu qua std::function).

Mục	Nội dung
Mã use case	UC08
Tên use case	Quản lý phiếu hỗ trợ kỹ thuật
Sinh viên thực hiện	Họ tên: Nguyễn Bá Thành Văn MSSV: ……
Lớp đối tượng (C++)	PhieuHoTro (kèm lớp menu PhieuHoTroMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maPhieu (string), maSTB (string), moTa (string), trangThai (int) (1: Moi, 2: Dang xu ly, 3: Hoan thanh), ngayTao (string)
Tên file dữ liệu	data/phieuhotro.txt
Định dạng một bản ghi	maPhieu|maSTB|moTa|trangThai|ngayTao
Tham chiếu tới (khóa ngoại)	SetTopBox (maSTB)
Được tham chiếu bởi	Không có
Chức năng Thêm (C)	Kiểm tra mã thiết bị STB có tồn tại trong kho không (stbTonTai); validate ngày tạo với đồng hồ hệ thống (<ctime>); thêm phiếu mới.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Xem danh sách phiếu hỗ trợ kỹ thuật, Tìm kiếm theo mã phiếu, mã thiết bị STB hoặc nội dung lỗi.
Chức năng Cập nhật (U)	Cập nhật mô tả sự cố và cập nhật tiến độ xử lý phiếu (Mới →Đang xử lý →Hoàn thành).
Chức năng Xóa (D)	Chặn xóa phiếu hỗ trợ nếu sự cố chưa được xử lý xong (trangThai != 3).
Kỹ thuật OOP áp dụng	Đóng gói, Kế thừa, Đa hình, Tích hợp thư viện thời gian hệ thống và xử lý chuỗi an toàn.

Mục	Nội dung
Mã use case	UC09
Tên use case	Quản lý hợp đồng IPTV
Sinh viên thực hiện	Họ tên: Đỗ Gia Huy MSSV: ……
Lớp đối tượng (C++)	HopDong (kèm lớp menu HopDongMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maHopDong (string), maKhachHang (string), goiDichVu (string), ngayKy, ngayBatDau,ngayKetThuc (string), donGiaThang (double), trangThai (string)
Tên file dữ liệu	data/hopdong.txt
Định dạng một bản ghi	maHopDong|maKhachHang|goiDichVu|ngayKy|ngayBatDau|ngayKetThuc|donGiaThang|trangThai
Tham chiếu tới (khóa ngoại)	KhachHang (maKhachHang)
Được tham chiếu bởi	HoaDon (maHopDong)
Chức năng Thêm (C)	Kiểm tra mã khách hàng; kiểm tra đơn giá không âm; thêm mới hợp đồng dịch vụ.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Xem toàn bộ danh sách hợp đồng đã ký kết, tìm kiếm hợp đồng theo mã hợp đồng, mã khách hàng hoặc gói cước.
Chức năng Cập nhật (U)	Cập nhật chu kỳ ngày tháng, gói cước, đơn giá và trạng thái hiệu lực của hợp đồng.
Chức năng Xóa (D)	Chặn xóa nếu hợp đồng đang có hóa đơn cước tham chiếu trong hệ thống (repoHoaDon).
Kỹ thuật OOP áp dụng	Trừu tượng hóa, Kế thừa, Đóng gói, Đa hình, Thiết kế phân tách Controller - Model chuẩn OOP.

Mục	Nội dung
Mã use case	UC10
Tên use case	Quản lý hoá đơn và thanh toán
Sinh viên thực hiện	Họ tên: Đỗ Gia Huy MSSV: ……
Lớp đối tượng (C++)	HoaDon (kèm lớp menu HoaDonMenu)
Lớp cơ sở kế thừa	Entity
Danh sách thuộc tính và ràng buộc	maHoaDon (string), maHopDong (string), kyCuoc (string), ngayLap, hanThanhToan (string), soTien (double), trangThaiThanhToan (string)
Tên file dữ liệu	data/hoadon.txt
Định dạng một bản ghi	maHoaDon|maHopDong|kyCuoc|ngayLap|hanThanhToan|soTien|trangThaiThanhToan
Tham chiếu tới (khóa ngoại)	HopDong (maHopDong)
Được tham chiếu bởi	Không có
Chức năng Thêm (C)	Kiểm tra mã hợp đồng có tồn tại trong hệ thống trước khi lập hóa đơn; kiểm tra số tiền cước.
Chức năng Xem, tìm kiếm, sắp xếp (R)	Xem danh sách hóa đơn, tìm kiếm theo mã hóa đơn, mã hợp đồng hoặc kỳ cước, tổng hợp doanh thu thực tế đã thu và tổng dư nợ cước chưa thu.
Chức năng Cập nhật (U)	Cập nhật kỳ cước, hạn thanh toán, số tiền cước phát sinh.
Chức năng Xóa (D)	Xóa hóa đơn với xác nhận an toàn từ người quản trị.
Kỹ thuật OOP áp dụng	Kế thừa Entity, Đóng gói, Đa hình, Tách biệt logic thanh toán nghiệp vụ vào Controller.